

The Retry Storm That Ate My Budget

Debugging the hidden cost loops in agentic AI

Karthik Akula - Sr. Technical Account Manager, AWS

Yamini Rathamsetty - Sr. Delivery Tech Lead, AWS



The dream we built

It felt like magic in the demo



A goal comes in. One orchestrator splits it into pieces, hands each to a specialist agent, and those agents call tools to get the work done.
Fast. Clever. Every light green.

Demos run once, on the happy path. Production does not.

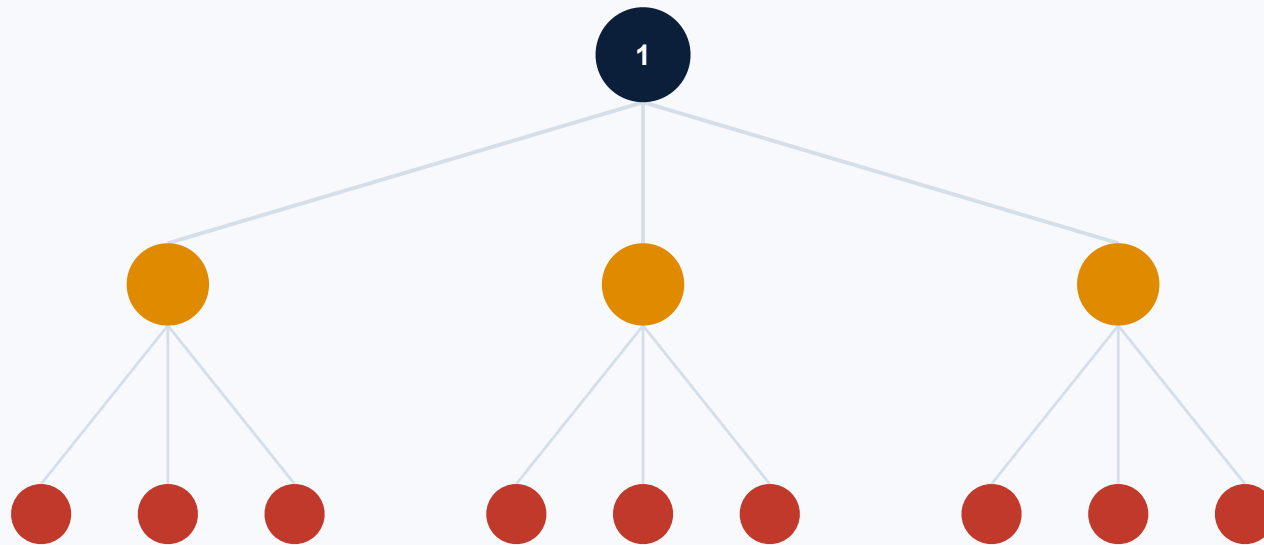
The fan-out

Cut off one head, it grows two

1 orchestrator

3 agents

9 sub-agents



1 request → 3 agents → 9 sub-agents → ...

Beautiful — but only while almost everything succeeds on the first try. Hold that thought.

Three layers of retry

Each one sensible. Nobody looked at them together.

Layer 1 — SDK botocore/boto3 max_attempts $\approx$ 3, with backoff. Invisible to you.

Layer 2 — App Our own resilience wrapper retries the SDK call. Reasonable. Also invisible.

Layer 3 — Agent On failure the agent re-reasons (a whole new model turn) and often spawns a helper.

$3 \times 3 \times 3$

and then an agent spawns and it starts again one level down.

The math

One slow dependency. One request. Then the multiplier.

1 request × 3 retry layers × 3 agents deep × re-reasoning

~\$300 / day

normal spend

~\$7,000

overnight. About 20× normal.

~4,000 calls

from a single request

A classic retry adds one pass. Agentic fan-out **multiplies** it. The dependency never fully failed. The bill did.

Figures illustrative, representative of the incident shape.

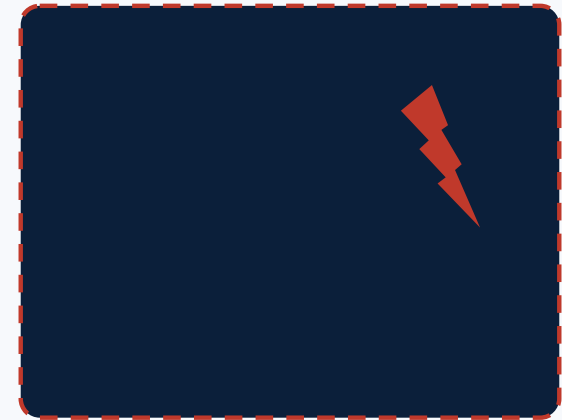
The trigger

Not an outage. Something worse.

A dependency that was slow and intermittently failing.

Up enough to not page anyone. Broken enough to fail ~1 call in 5.

One in five sounds survivable — until every layer decides to try harder, all at once, across dozens of agents.



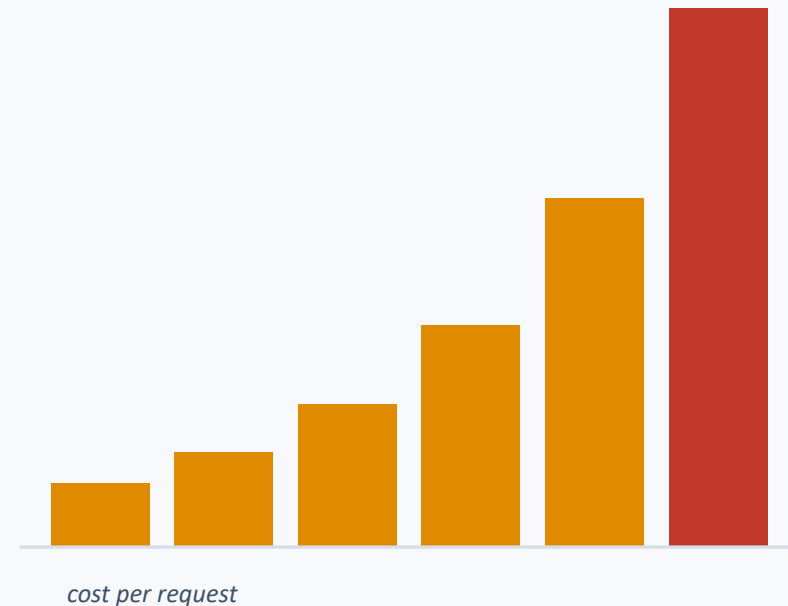
~1 in 5 fails

The silent compounding

Retries stop adding. They start multiplying.

A failed tool call → a new agent turn → tokens burned re-reasoning → the call re-issued → a helper agent spawned → its own retries...

Tokens climb from re-reasoning. Infra climbs from re-invocation. Both at once — and all of it looks like healthy activity.



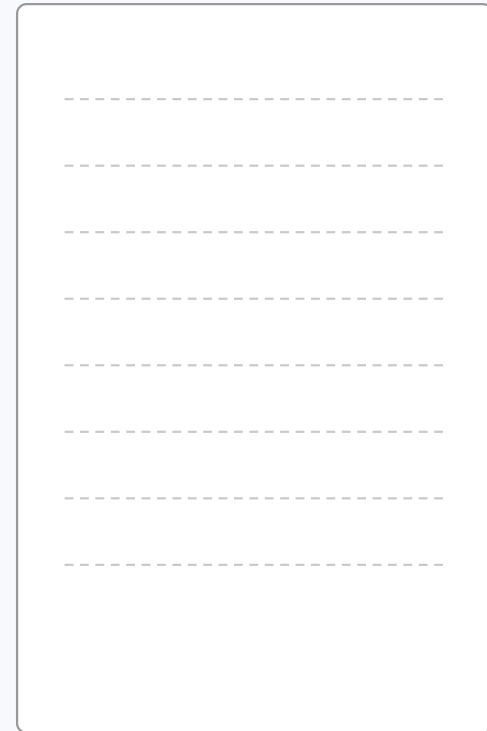
The bill

The line went vertical overnight

Multiples of normal spend — tokens AND infra.

No traffic spike. No new deploy. No incident. No alarm fired — because nothing technically failed.

It stopped being a cost problem and became a mystery.



The hunt

No single agent looked guilty

The cost was smeared across dozens of agents, each doing a small, reasonable amount of extra work. Error rate looked fine — retrying and eventually succeeding reads as success.

Breakthrough: stop counting errors. Count volume — agent turns and tool calls per request. One request had fired thousands of downstream calls. That was the fingerprint.

The villain revealed



*Three ordinary things. Dangerous only in combination.
The system isn't broken. It's working exactly as
designed. The design is quietly, expensively wrong.*

Locally sensible, globally expensive

Every decision was reasonable. No one owned the sum.

1

Each layer was added by a different team, for a good reason.

SDK retries, an app wrapper, agent re-reasoning. All defensible on their own.

2

No single owner looked at retries end to end.

Three local optimums stacked into one global failure mode.

3

The fix is an owner, not a scapegoat.

Give the whole request path one budget and one witness. Make the sum someone's job.

Why this is an agentic problem

Classic retry

Re-runs the same cheap code path.
Cost is roughly linear — one extra pass.

Agentic retry

Re-runs model reasoning — the most expensive thing you do.
And it can decide to create more work on its own.

Autonomy is the feature we want. It's also what turns a small failure into a budget event.

Fix 1: Retry budgets & depth limits



Budget every request

A hard ceiling on total agent turns, tool calls, and spawns. Not just a timeout.



Cap fan-out depth

An orchestrator may spawn workers; a worker 3 levels down may not spawn its own tree.



Fail loud

When the budget is hit, stop and fail visibly. Far better than succeeding at 100× the cost.

Fix 2: Circuit breakers & idempotency



Circuit breakers

When a dependency starts failing, trip the breaker and back off (half-open to test recovery).



Collapse the layers

Pick ONE layer to own retries (SDK, app, or agent) and turn the others off.



Idempotency keys

Give repeated work a key so a retry returns the cached result instead of paying again.

Fix 3: Cost as a first-class signal

The fix that would have saved us the whole ordeal



Emit tokens & tool-calls per request

A live metric, right next to latency and errors.



Trace the full agent tree

See the fan-out, not just individual calls.



Alert on volume, not just failures

If one request triggers thousands of calls, that should page someone.

What we instrumented

Fix 3, made concrete. The three signals we added.



Per-request cost tags

Tag every request with tokens and tool-calls. Emit them live, like latency and errors.



Spans across the agent tree

One trace per request, spanning orchestrator, every sub-agent, and every tool call.

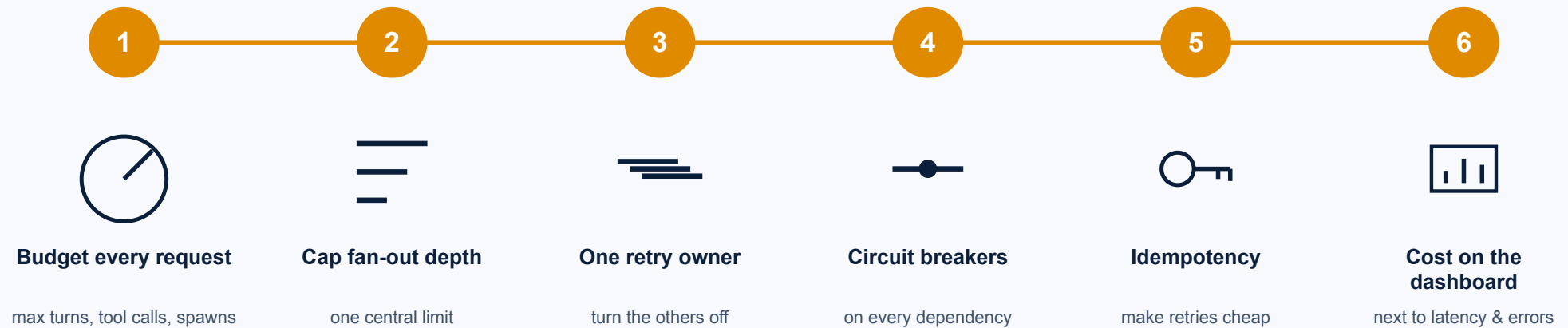


Alert on volume, not just errors

Page when one request crosses a call or token ceiling. Retrying-to-success no longer hides it.

The playbook

The checklist we wish someone had handed us before we shipped



Watch the storm before it watches your budget

Agents don't fall over — they get quietly, enthusiastically busy, and hand you the bill later. Give your agents a budget, a limit, and a witness.

Now go check what your agents are doing at 2 a.m.